Object-Oriented Software Engineering
Using UML, Patterns, and Java

Chapter 2, Modeling with UML: Review Session

Outline for this Week

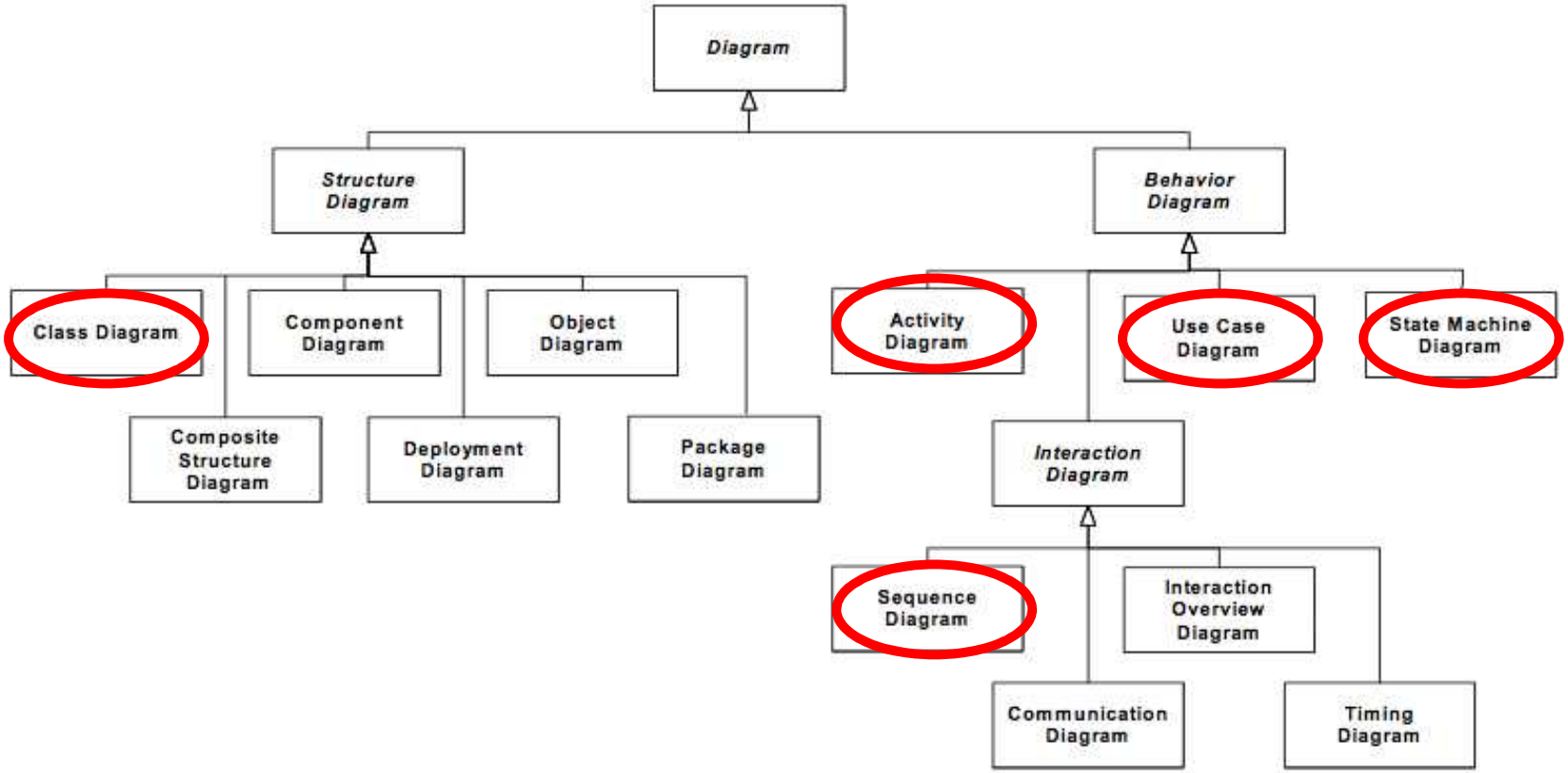
- Last Lecture: Modeling Functions, Structure and Behavior
  - Use case diagrams
  - Class diagrams
  - Sequence diagrams, State chart diagrams, Activity diagrams
- Today we review these concepts
  - Review: Why UML?
  - Review: Diagram notations

We use Models to describe Software Systems

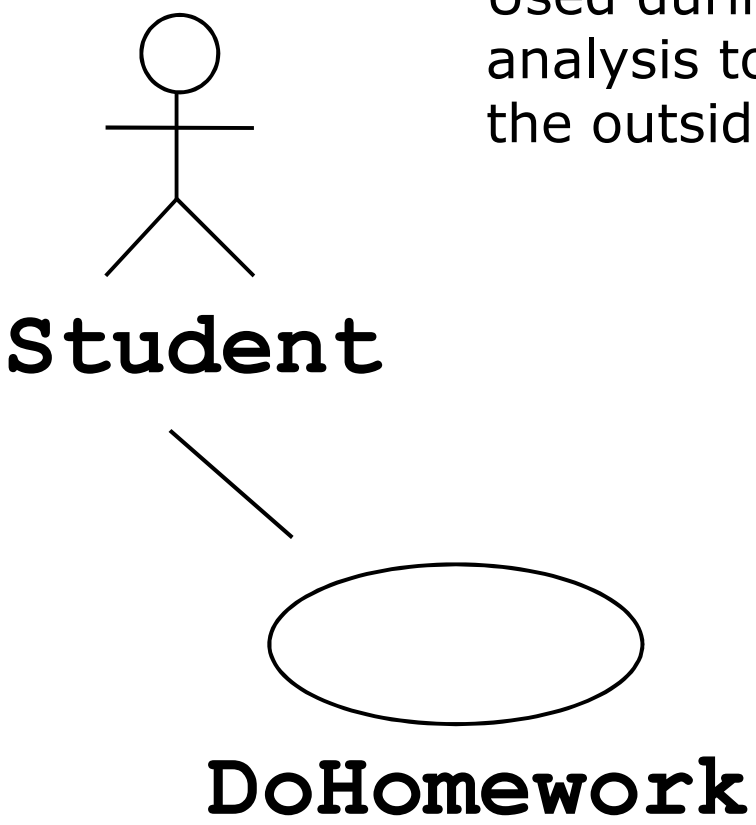
- System model: Object model + functional model + dynamic model

- Object model: What is the structure of the system?
  - UML Notation: Class diagrams
- Functional model: What are the functions of the system?
  - UML Notation: Use case diagrams
- Dynamic model: How does the system react to external events?
  - UML Notation: Sequence, State chart and Activity diagrams

Another view on UML Diagrams



Review of Use Case Diagrams: 3 Important Terms



Used during requirements elicitation and analysis to represent behavior visible from the outside of the system

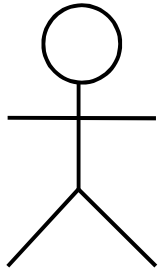
An ***actor*** represents a role, that is, a type of user of the system

A ***use case*** represents a class of functionality provided by the system

Use case model:

The set of all use cases that completely describe the functionality of the system.

Actor



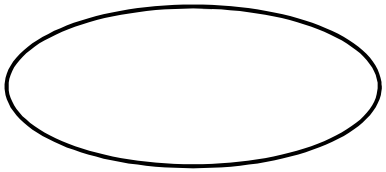
Student

- An actor is a model for an external entity which interacts with the system:
  - EndUser, Administrator
  - External system (Another system)
  - Physical environment (e.g. Weather)
- An actor has a unique name and an optional description
- Examples:
  - **Student**: A studying person
  - **Teaching Assistant**: Member of teaching staff who supports the instructor.
  - **Random Number generator**

Optional Description

Name

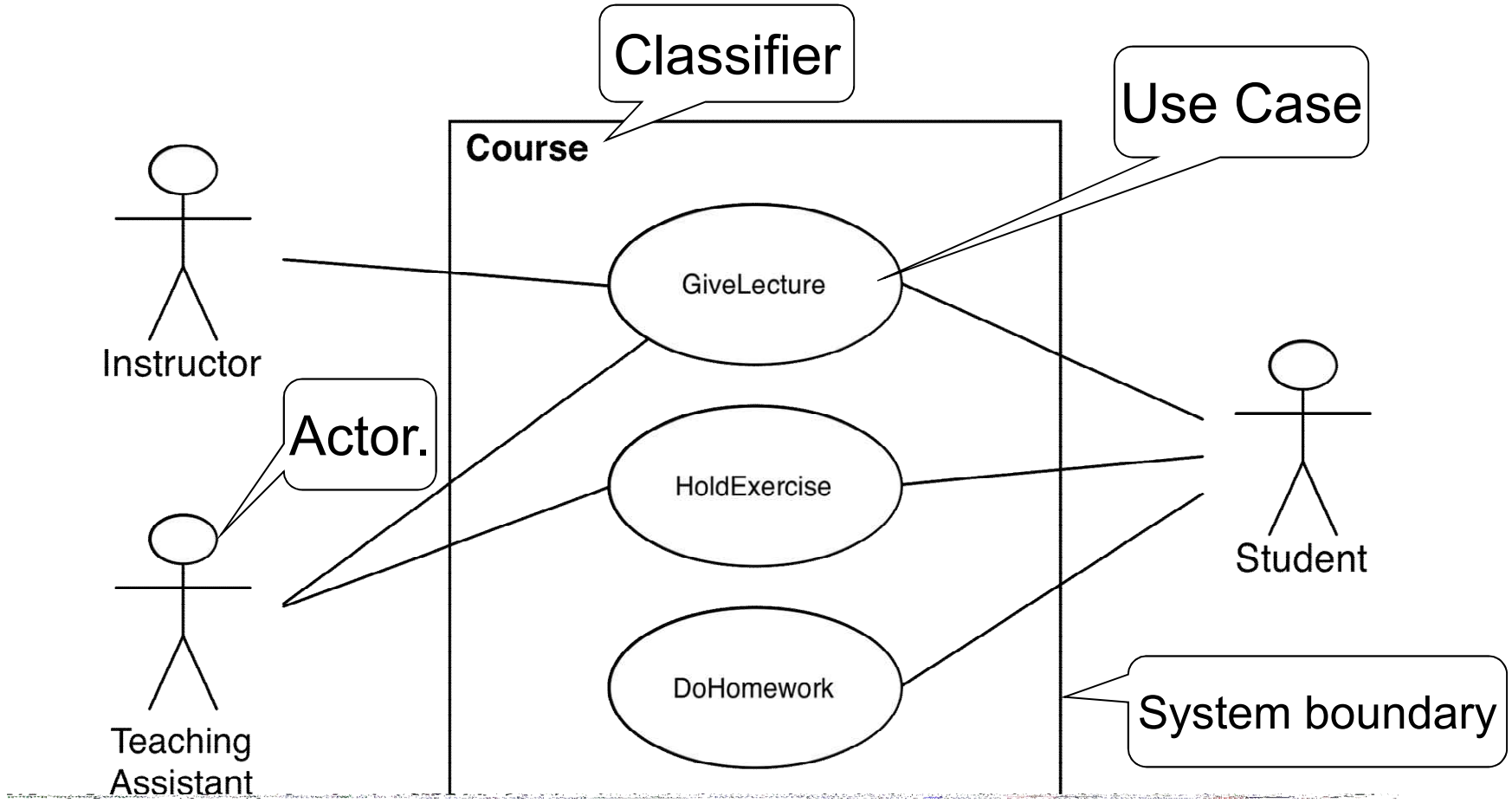
Use Case



DoHomework

- A use case represents a class of functionality provided by the system
- Use cases can be described textually, with a focus on the event flow between actor and system
- The textual use case description consists of 6 parts:
  1. Unique name
  2. Participating actors
  3. Entry conditions
  4. Exit conditions
  5. Flow of events
  6. Special requirements.

Use Case Model

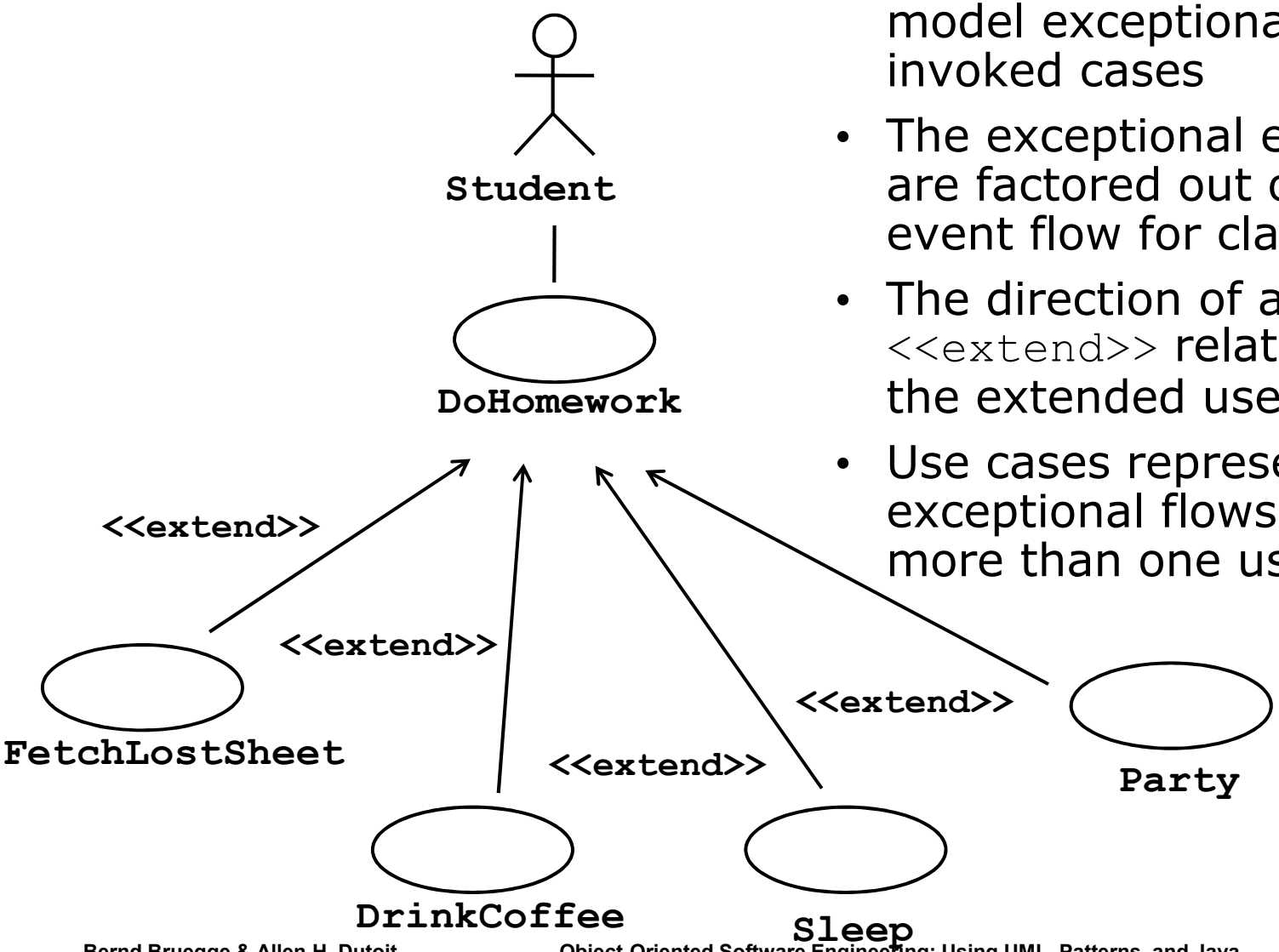


Use case diagrams represent the functionality of the system from user's point of view

Uses Cases can be related

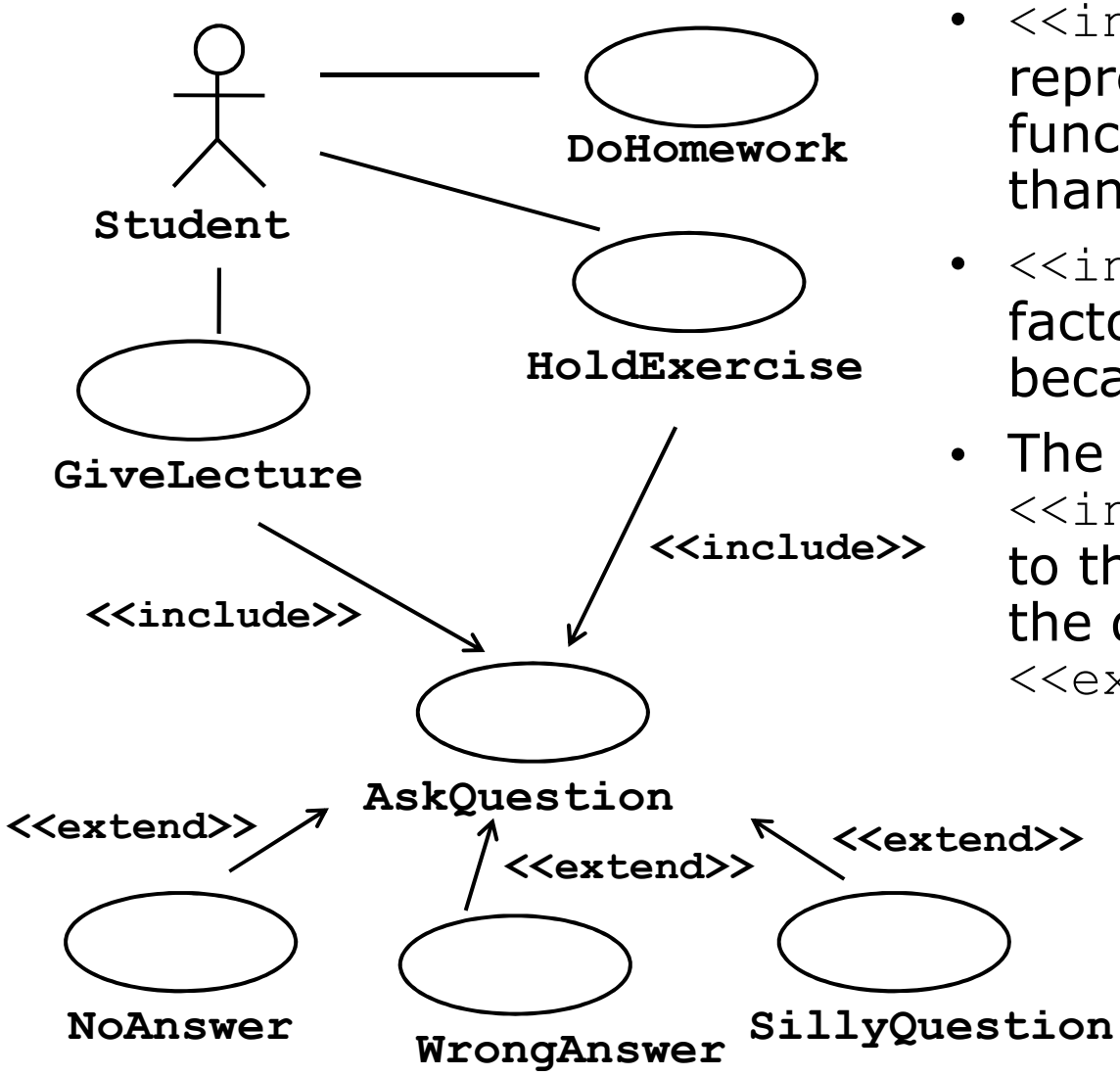
- Extend Relationship
  - To represent seldom invoked use cases or exceptional functionality
- Include Relationship
  - To represent functional behavior common to more than one use case.

The <<*extend*>> Relationship



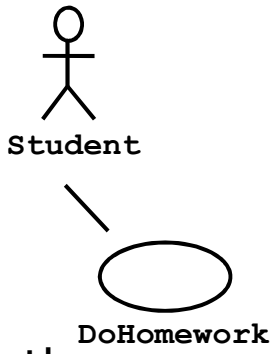
- <<extend>> relationships model exceptional or seldom invoked cases
- The exceptional event flows are factored out of the main event flow for clarity
- The direction of an <<extend>> relationship is to the extended use case
- Use cases representing exceptional flows can extend more than one use case.

The *<<include>>* Relationship



- <<include>> relationship represents common functionality needed in more than one use case
- <<include>> behavior is factored out for reuse, not because it is an exception
- The direction of a <<include>> relationship is to the using use case (unlike the direction of the <<extend>> relationship).

Textual Use Case Description Example



1. Name: `DoHomework`

2. Participating actor:
`Student`

3. Entry condition:

- `Student` received exercise sheet
- `Student` is in good health

4. Exit condition:

- `Student` delivered solution

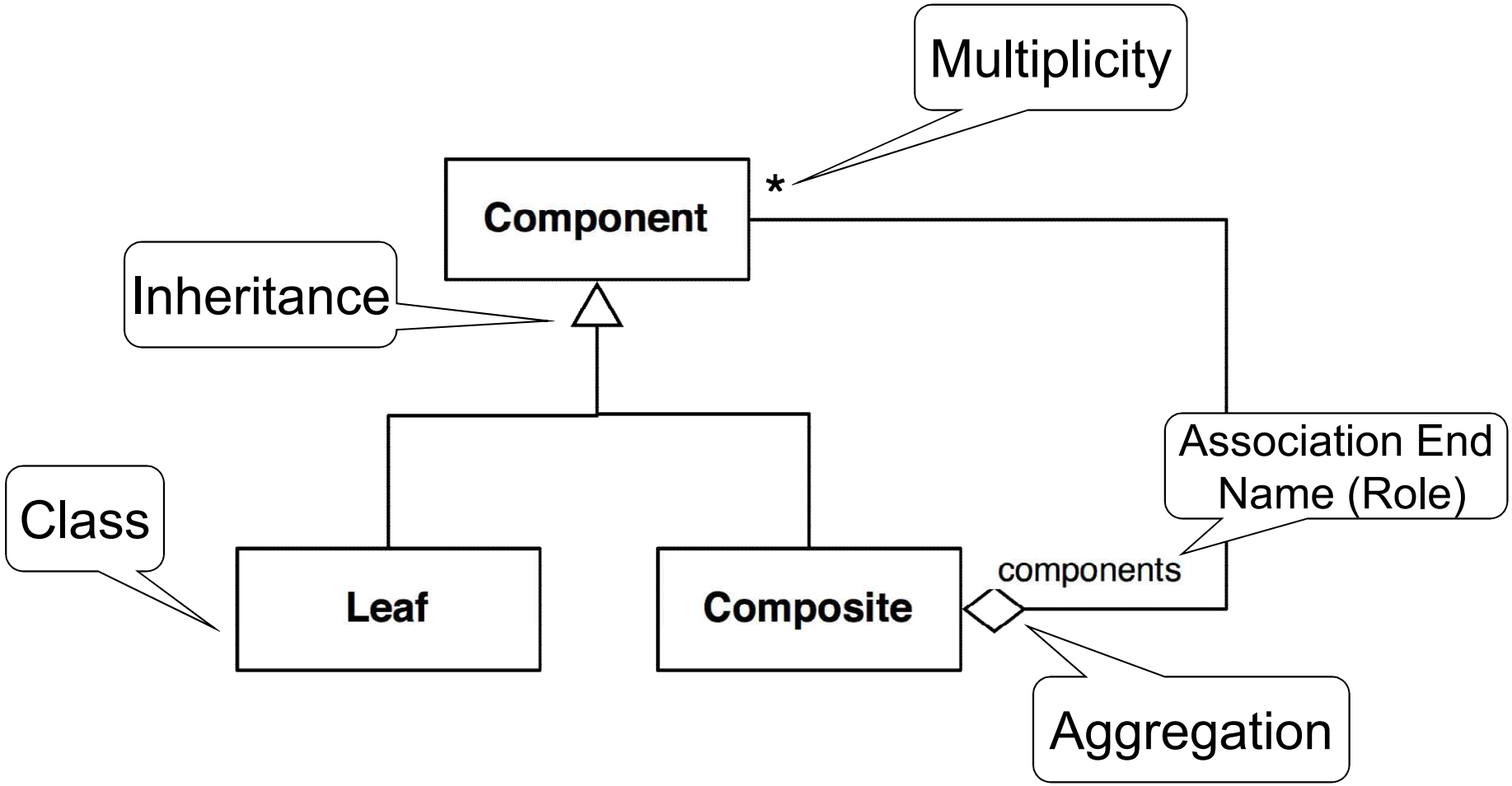
5. Flow of events:

1. `Student` fetches the exercise sheet
2. `Student` reads through the assignments
3. `Student` processes the assignments and types the solution in his Computer.
4. `Student` prints out the solution
5. `Student` delivers the solution in the following exercise

6. Special requirements:

None.

Review of Class Diagrams



Class diagrams represent the structure of the system

Actor vs Class vs Object

- **Actor**
  - An entity outside the system to be modeled, interacting with the system ("Passenger")
- **Class**
  - An abstraction modeling an entity in the application or solution domain
  - The class is part of the system model ("User", "Ticket distributor", "Server")
- **Object**
  - A specific instance of a class ("Joe, the passenger who is purchasing a ticket from the ticket distributor").

Many-to-many Associations



- A stock exchange lists many companies.
- Each company is identified by a ticker symbol

From Problem Statement To Object Model

Problem Statement: A stock exchange lists many companies. Each company is uniquely identified by a ticker symbol

Class Diagram:

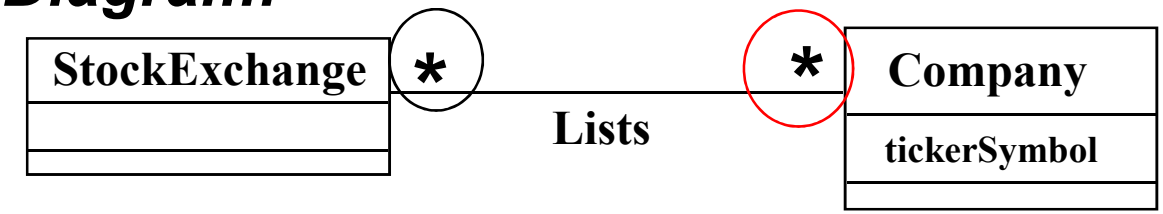


From Problem Statement to Code

Problem Statement: A stock exchange lists many companies

Each company is identified by a ticker symbol

Class Diagram:



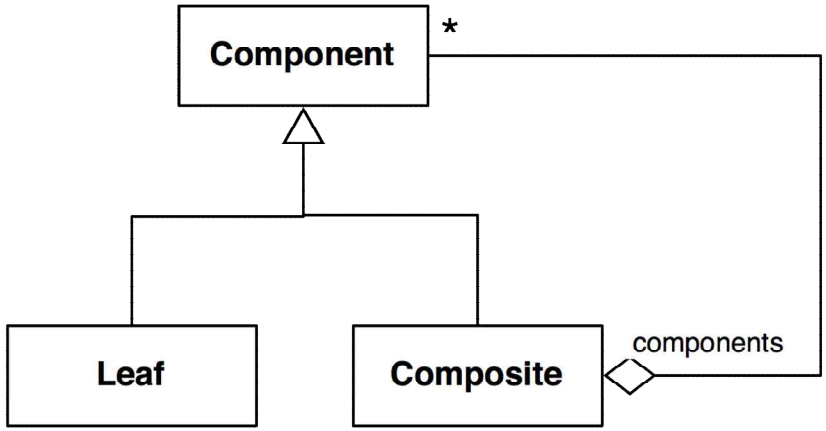
Java Code

```
public class StockExchange
{
 private Vector m_Company = new Vector();
};

public class Company
{
 public int m_tickerSymbol;
 private Vector m_StockExchange = new Vector();
};
```

Associations are mapped to attributes.

Code Generation from UML to Java I

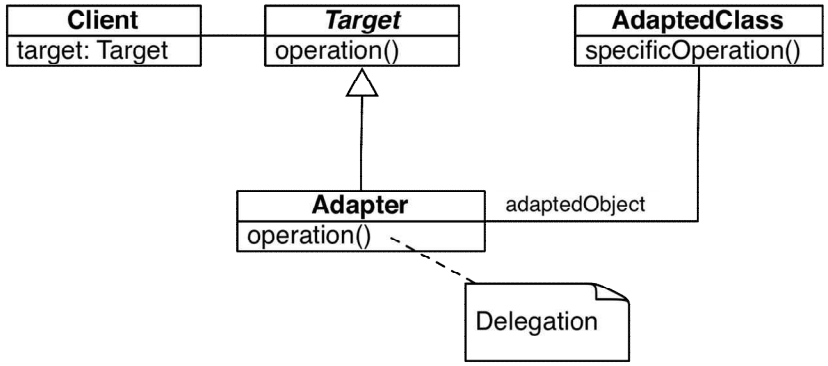



```
public class Component{  }

public class Leaf extends
    Component{  }

public class Composite extends
    Component{
    private Collection<Component>
    components;
    …
}
```

Code Generation from UML to Java II



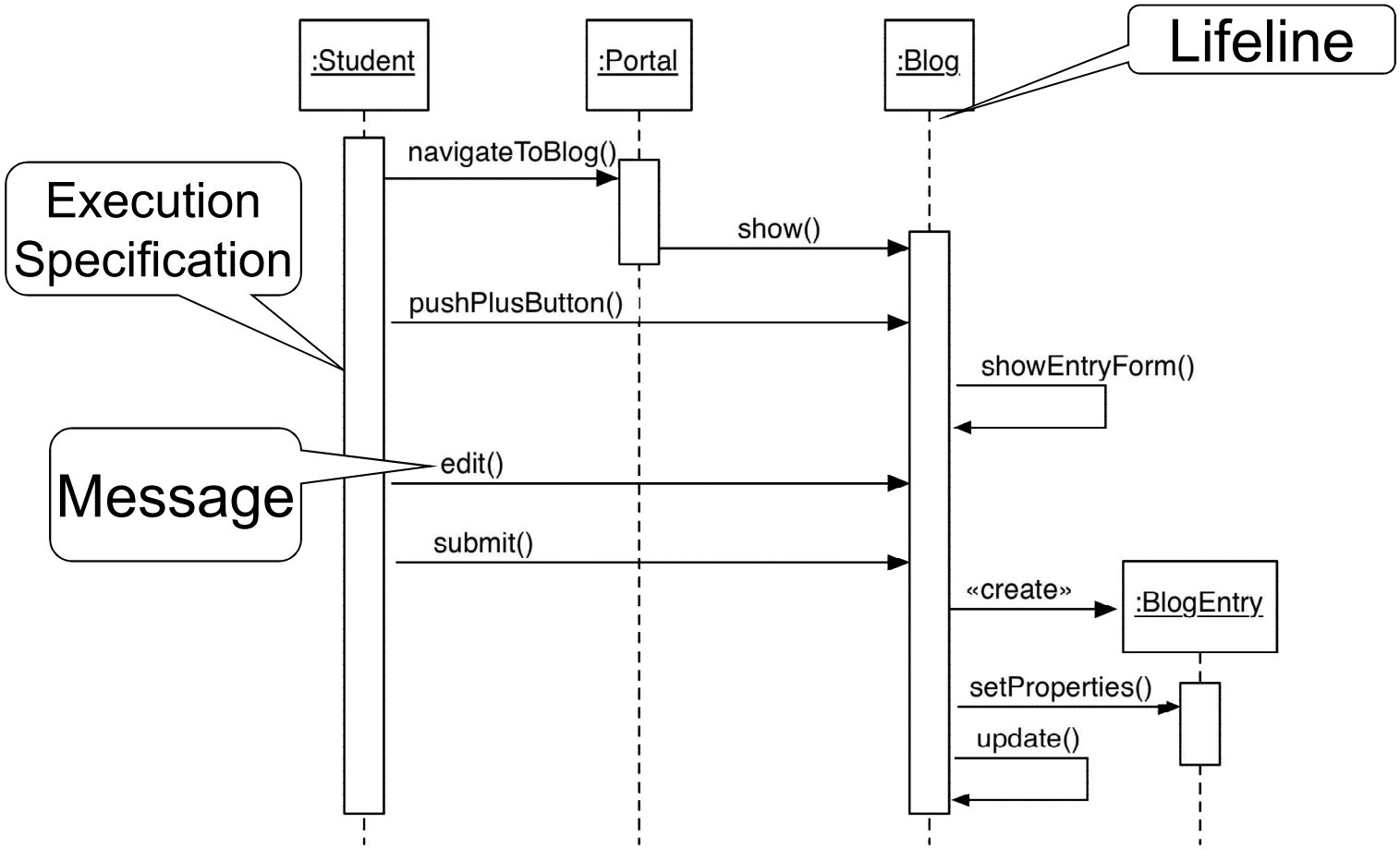

```
public abstract class Target{
    public … operation(); }

public class Adapter extends Target {
    private AdaptedClass adaptedObject;
    public … operation(){
            adaptedObject.specificOperation();
    }
```

Where are we?

- ✓ Review functional modeling
  - ✓ Use case diagram
- ✓ Review object modeling
  - ✓ Class diagram
  - ✓ Generating source code from class diagrams
- ➢ Review dynamic modeling
  - ➢ Sequence diagram
  - • Statechart diagram

Sequence diagram: Basic Notation



Sequence diagrams represent the behavior of a system as messages (“interactions”) between *different objects.*

Lifeline and Execution Specification

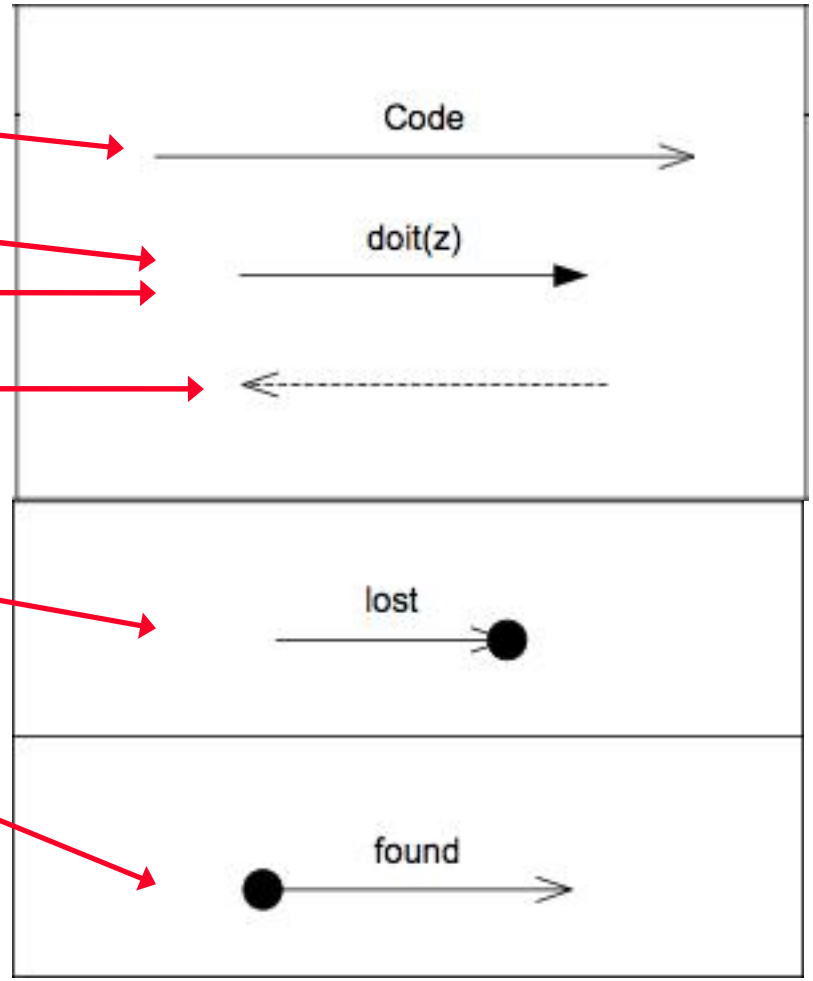
- A **lifeline** represents an individual participant (or object) in the interaction
- A lifeline is shown using a symbol that consists of a rectangle forming its "head" followed by a vertical line (which may be dashed) that represents the lifetime of the participant
- An **execution specification** specifies a behavior or interaction within the lifeline
- An execution specification is represented as a thin rectangle on the lifeline.

Messages

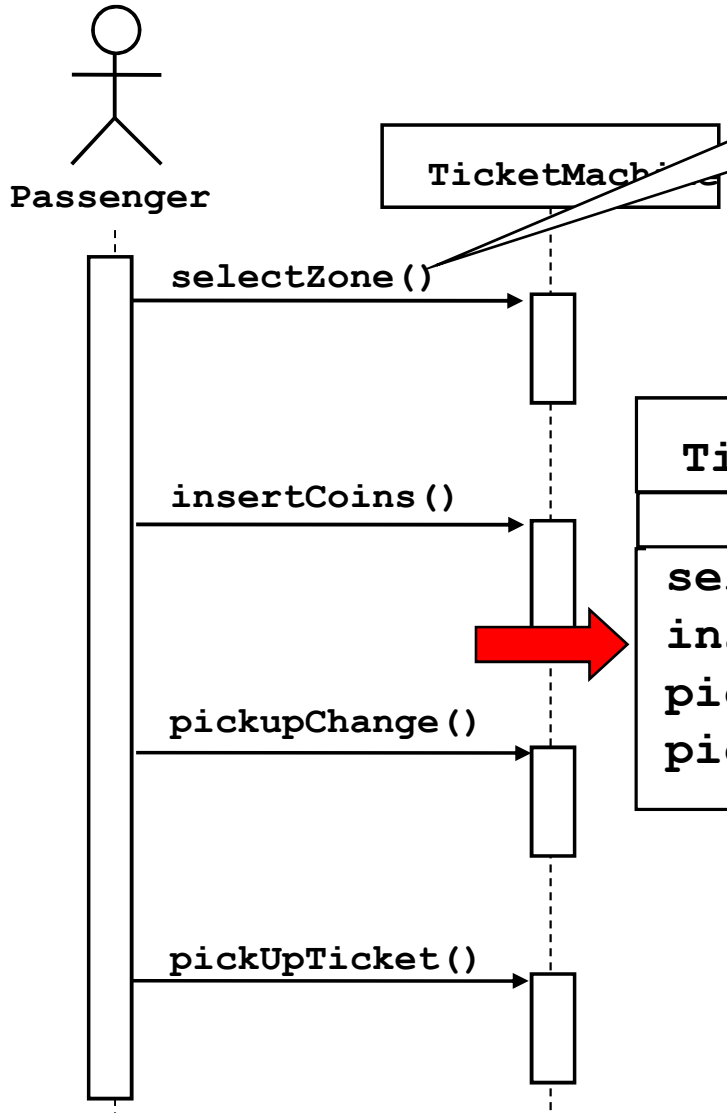
- Define a particular communication between lifelines of an interaction
- Examples of communication
  - raising a signal
  - invoking an operation
  - creating or destroying an instance
- Specify (implicitly) sender and receiver
- are shown as a line from the sender to the receiver
- Form of line and arrowhead reflect message properties

Message Types

- Asynchronous
- Synchronous
- Call and Object creation
- Reply
- Lost
- Found



Sequence Diagrams



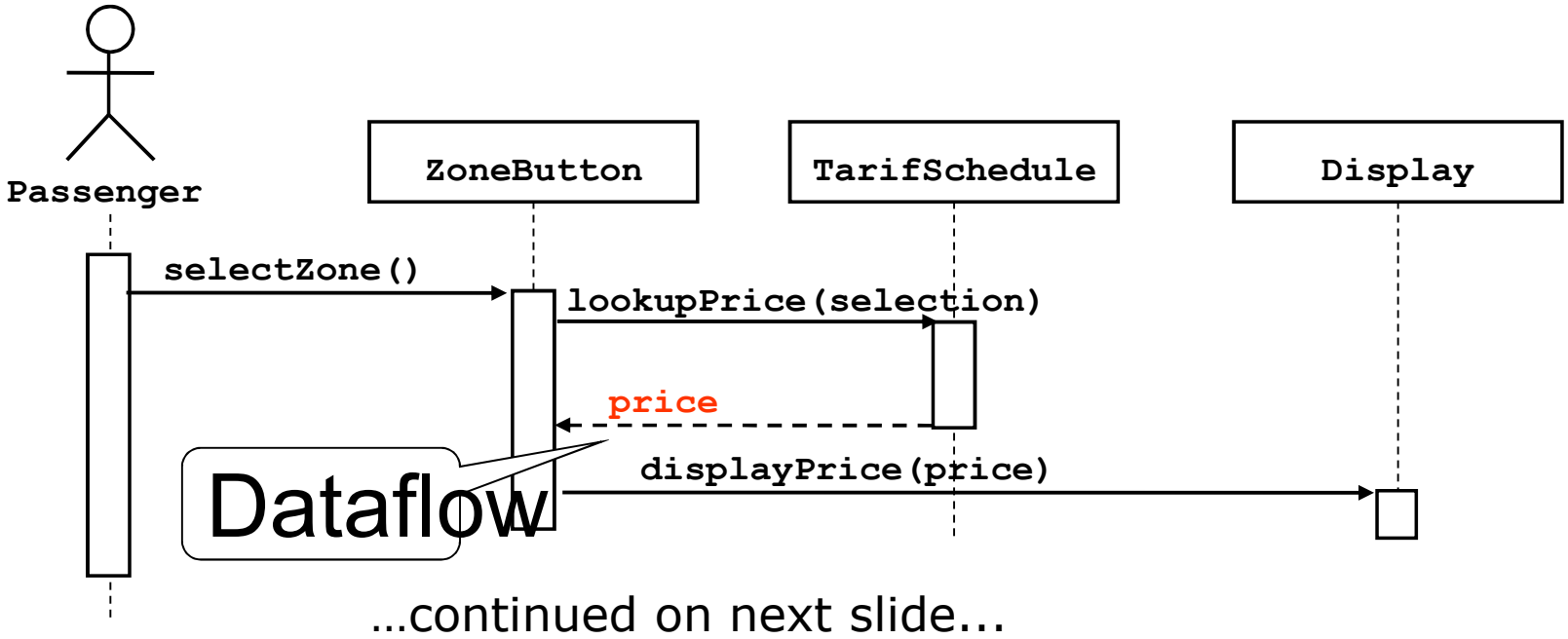
Focus on Controlflow

- Used during analysis
  - To refine use case descriptions
  - to find additional objects ("participating objects")
- Used during system design
  - to refine subsystem interfaces
  - to refine ***Classes*** and ***Associations***
  - to refine ***Operations*** and ***Signatures***

Messages -> Operations on participating Object

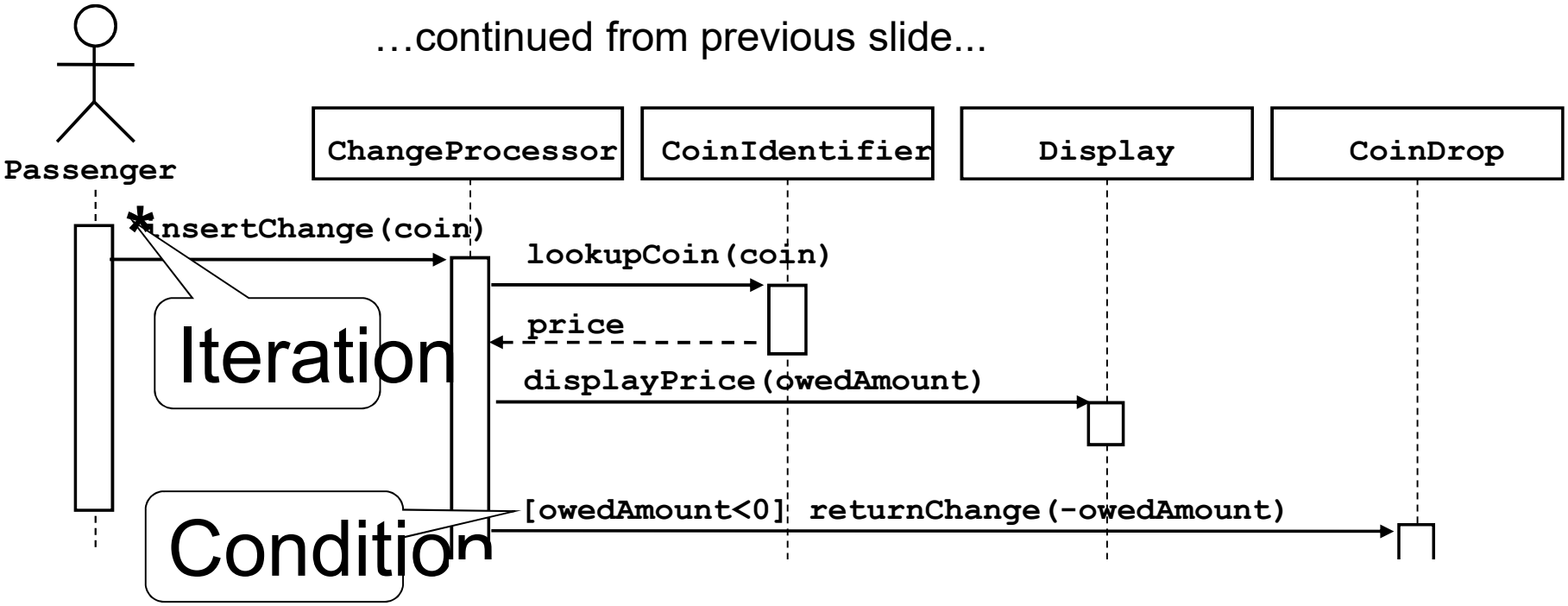
- ***Messages*** are represented by arrows
- ***Activations*** are represented by narrow rectangles.

Sequence Diagrams can also model the Flow of Data



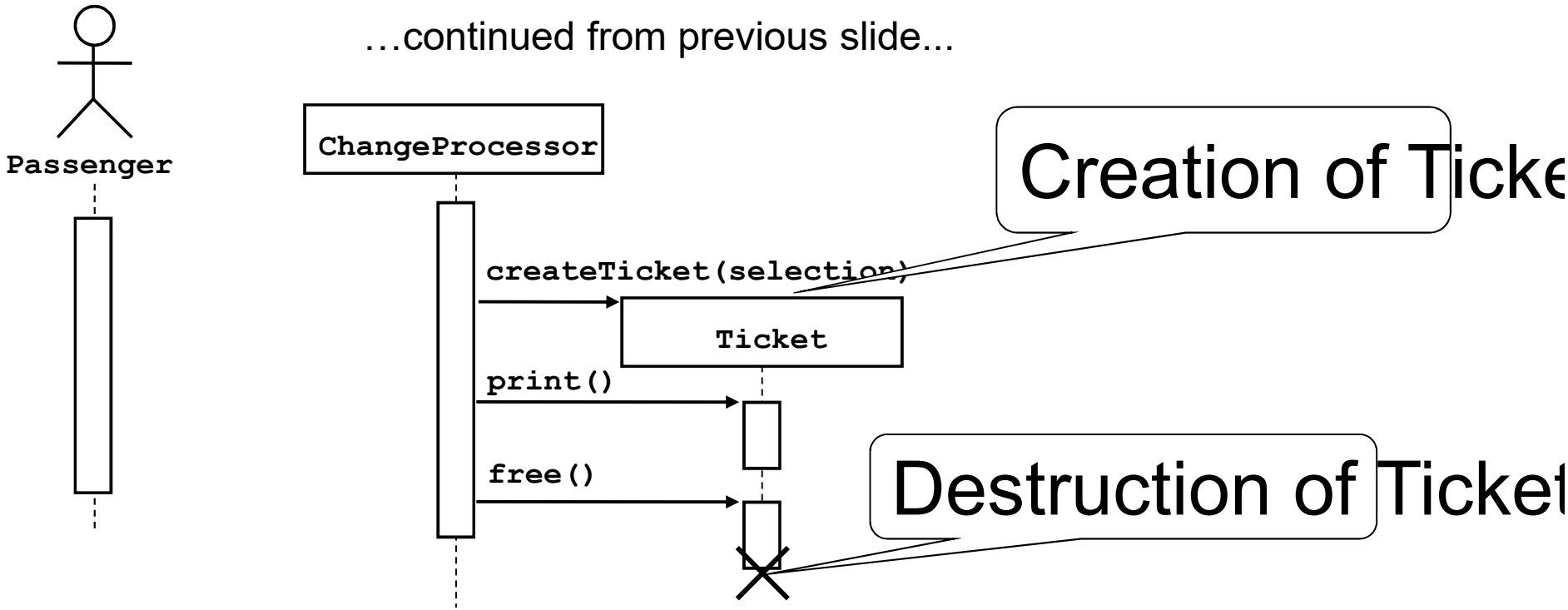
- The source of an arrow indicates the activation which sent the message
- Horizontal dashed arrows indicate data flow, for example return results from a message

Sequence Diagrams: Iteration & Condition



- Iteration is denoted by a * preceding the message name
- Condition is denoted by boolean expression in [ ] before the message name

Creation and destruction

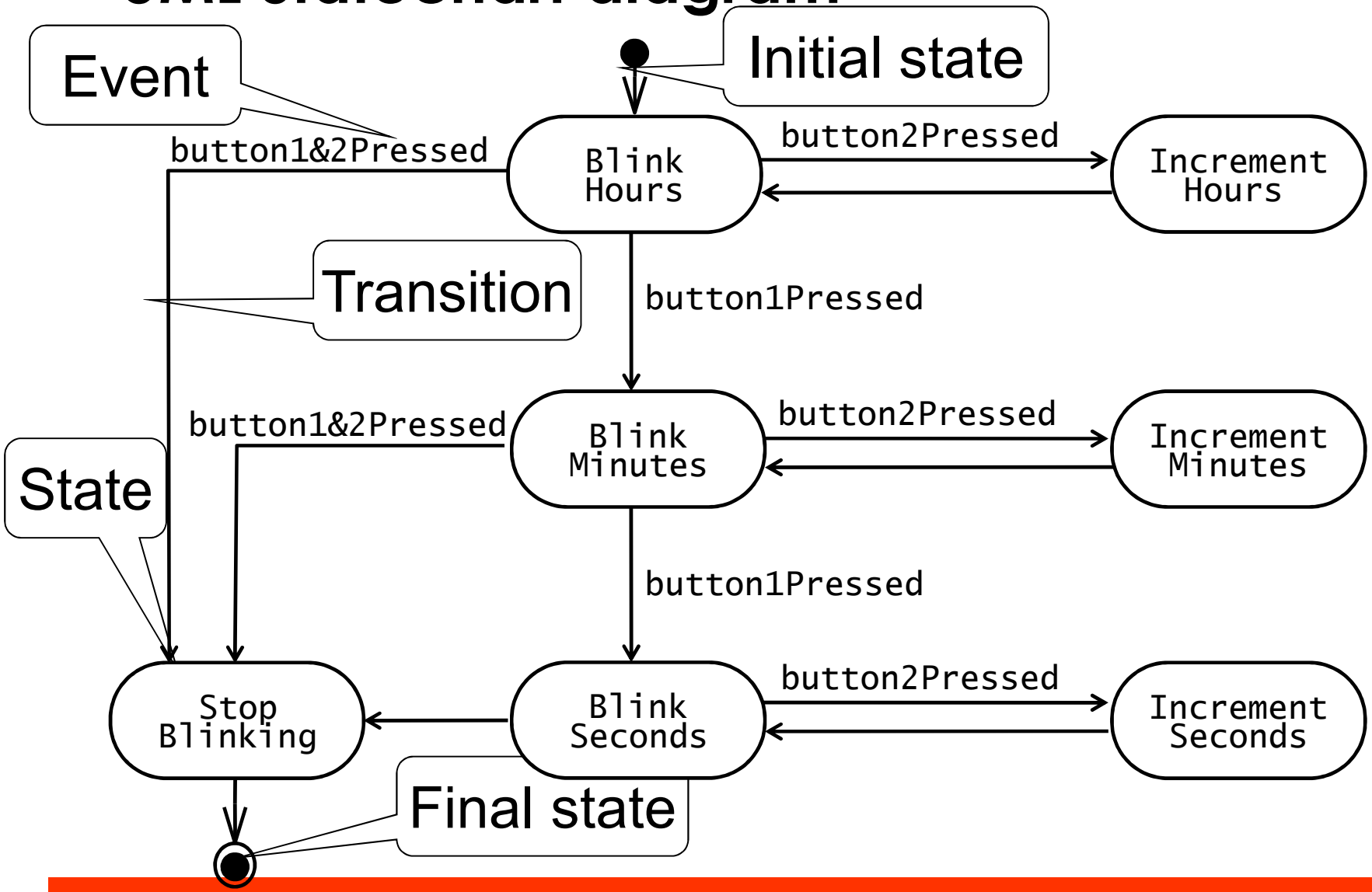


- Creation is denoted by a message arrow pointing to the object
- Destruction is denoted by an X mark at the end of the destruction activation
  - In garbage collection environments, destruction can be used to denote the end of the useful life of an object.

Sequence Diagram Properties

- UML sequence diagram represent *behavior in terms of interactions*
- Useful to identify or find missing objects
- Time consuming to build, but worth the investment
- Complement the class diagrams (which represent structure).

UML Statechart diagram



Represents behavior of *a single object* with interesting dynamic behavior.